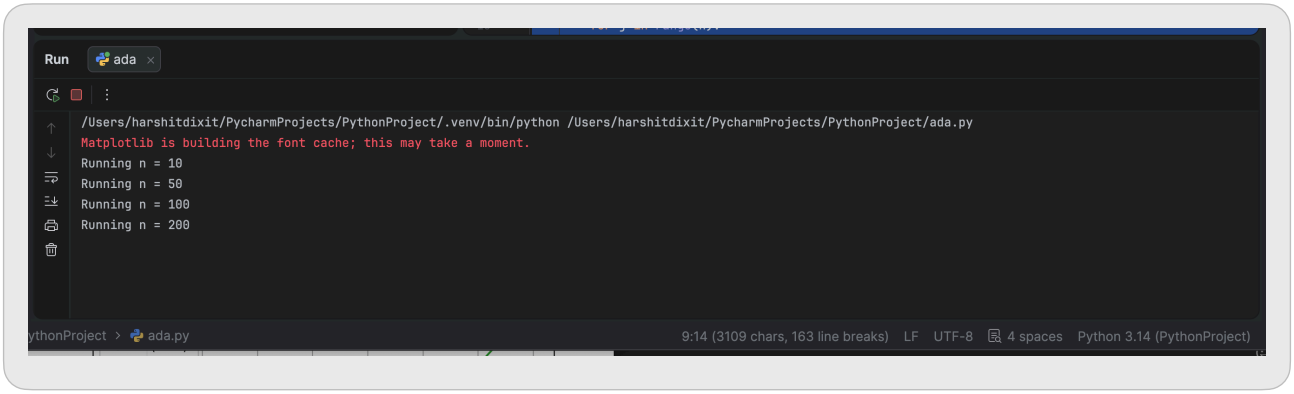
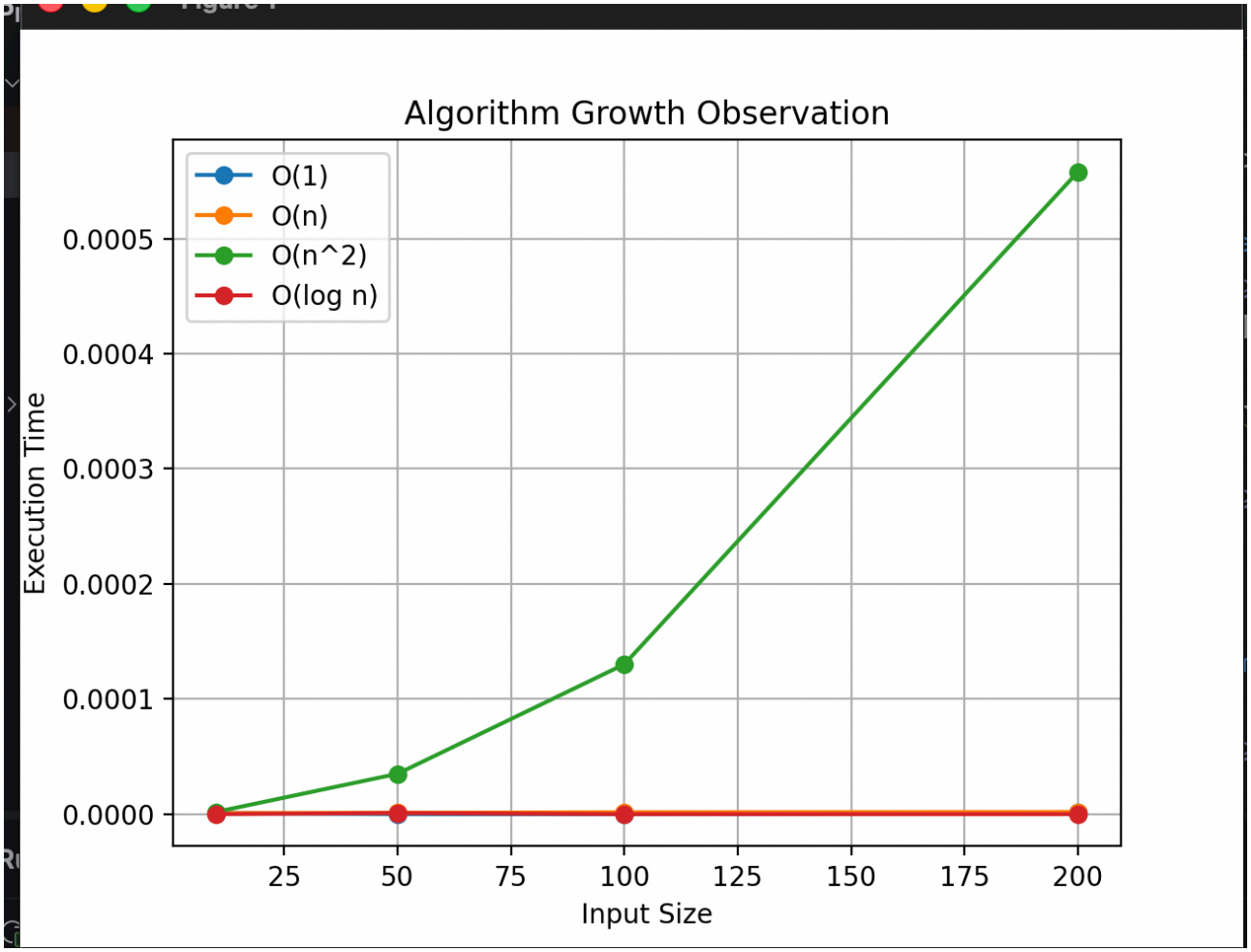
```
import time
import random
import matplotlib.pyplot as plt

def constant_time(n):
    arr=list(range(n))
    return arr[0]

def linear_time(n):
    s=0
    for i in range(n):
        s+=i
    return s

def quadratic_time(n):
    c=0
    for i in range(n):
        for j in range(n):
            c+=1
    return c

def logarithmic_time(n):
    c=0
    while n>1:
        n=n//2
        c+=1
    return c

n_values=[10,50,100,200]

algorithms={
"O(1)":constant_time,
"O(n)":linear_time,
"O(n^2)":quadratic_time,
"O(log n)":logarithmic_time
}

time_results={k:[] for k in algorithms}

for n in n_values:
    print("Running n =",n)
    for name,func in algorithms.items():
```

```
        start=time.time()
        func(n)
        end=time.time()
        time_results[name].append(end-start)

for name in time_results:
    plt.plot(n_values,time_results[name],marker="o",label=name)

plt.xlabel("Input Size")
plt.ylabel("Execution Time")
plt.title("Algorithm Growth Observation")
plt.legend()
plt.grid()
plt.show()

def linear_search(arr,target):
    for i in range(len(arr)):
        if arr[i]==target:
            return i
    return -1

def binary_search(arr,target):
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==target:
            return mid
        elif arr[mid]<target:
            low=mid+1
        else:
            high=mid-1
    return -1

sizes=[100,500,1000,2000]

linear_times=[]
binary_times=[]

for n in sizes:
    arr=sorted(random.sample(range(1,10000),n))
    target=arr[-1]
```

```
    start=time.time()
    linear_search(arr,target)
    linear_times.append(time.time()-start)

    start=time.time()
    binary_search(arr,target)
    binary_times.append(time.time()-start)

plt.plot(sizes,linear_times,marker="o",label="Linear Search")
plt.plot(sizes,binary_times,marker="o",label="Binary Search")

plt.xlabel("Array Size")
plt.ylabel("Execution Time")
plt.title("Linear Search vs Binary Search")
plt.legend()
plt.grid()
plt.show()

call_counter=0

def fibonacci_recursive(n):
    global call_counter
    call_counter+=1
    if n<=1:
        return n
    return fibonacci_recursive(n-1)+fibonacci_recursive(n-2)

def fibonacci_dp(n):
    fib=[0]*(n+1)
    if n>0:
        fib[1]=1
    for i in range(2,n+1):
        fib[i]=fib[i-1]+fib[i-2]
    return fib[n]

print("Fibonacci Comparison")

for n in [5,10,15]:
    call_counter=0

    start=time.time()
```

```
    fibonacci_recursive(n)
    rec=time.time()-start
    calls=call_counter

    start=time.time()
    fibonacci_dp(n)
    dp=time.time()-start

    print("n =",n,"Recursive Time =",rec,"DP Time =",dp,"Calls =",calls)

count1=0

def recurrence1(n):
    global count1
    count1+=1
    if n<=1:
        return 1
    return recurrence1(n//2)+n

count2=0

def recurrence2(n):
    global count2
    count2+=1
    if n<=1:
        return 1
    return recurrence2(n//2)+recurrence2(n//2)+n

print("Recurrence Results")

for n in [8,16,32]:
    count1=0
    recurrence1(n)
    print("T(n)=T(n/2)+n , n =",n,"calls =",count1)

    count2=0
    recurrence2(n)
    print("T(n)=2T(n/2)+n , n =",n,"calls =",count2)
```





GitHub = https://github.com/Kesshavv/ada-assignment-1